NLP Прямая / плоскость / гиперплоскость

Регрессия: прибл к У

Классиф: разделяющая Х одна часть У другая

Логист. регр

Получаем число → ≥ порог $\xrightarrow{да}$ 1

$\xrightarrow{нет}$ 0

$$y = \begin{cases} 1, & \text{if } p\left(a + b_1 \cdot f_1(x) + b_2 \cdot f_2(x) + \dots + b_n f_n(x)\right) \\ 0, & \text{else} \end{cases}$$

Вычисл. вер-ть принадл к классу

Обучение:

ф-ция правдоподобия - такая

|| чтобы

плотность вер-ти совместного

наблюд. вер-ть в выборке

$$L = p(y_1|\theta) \cdot p(y_2|\theta) \dots \cdot p(y_k|\theta)$$

Чтобы не учитывать а складывать

делаем Log

$$L^* = \ln(L(y_1, y_2, \dots y_n)) \to max$$

$$\sum_{k=1}^{k} \left[y_k \ln(p_k) + (1 - y_k) \cdot \ln(1 - p_k) \right]$$

алг оптимиз

$$L^* \xrightarrow[\Theta]{} \max, \quad \Theta = (a, b_1, \dots, b_n)$$

$$P_k \sim \frac{1}{1+e^{-a+b_1 f_1(x_k) - \dots - b_n f_n(x_k)}}$$

- это метод градиентного подъема

Градиент:

$$\nabla L^*(\Theta) = \frac{\partial L^*}{\partial \Theta} = \left(\frac{\partial L^*}{\partial a}, \frac{\partial L^*}{\partial b_1}, \dots, \frac{\partial L^*}{\partial b_n} \right)$$

Начин. со случ знач

Шаг: → новое знач. вектора Θ

$$\Theta^{[j+1]} = \Theta^{[j]} + \alpha^{[j]} \nabla L^*(\Theta^{[j]})$$

α - длина шага в напр. град.

в-р

→ проверка условия останова

$$|\nabla L^*(\Theta^{[j+1]})| \le \varepsilon \text{ по } \Theta = \Theta^{[j+1]}$$

да → нашли

нет

α лучше менять в ходе
машинного обучения

↳ Задача оптимизации одного
по α

зная вектор ∇
какой шаг сделать?

$$\theta^{[j+1]} + \alpha^{[j]} \cdot \nabla L^*(\theta^{[j]}) \xrightarrow[\alpha^{[j]}]{} \max$$

Представьте что
L^* это что-то типа наград
для которого мы
уже знаем направл

Имея параболу на 3х точках
$\alpha_1 = 0.1$ ← математически можем построить
0.001 параболу и найти
0.0001 ее экстремум
Но никто не гарантир. что
мы будем около оси

Метод бренча / бренч
Делаем несколько чанов
до тех пор пока параметр
не станет хорошим
(?)

Numeric
Science
M C

Нашли все параметры
к-рые max L*
Но если мы сделаем это
сделаем то получим overfitting
Потеряем способн. к обобщ

↓
она есть если качество
работ на тестовой выборке
не сильно отличается
от того что на train

?: независим. test или нет
↙ ↘
если да то предполагаем
судьи наших

надеемся что
test не реальные
данные не сильно
отличается от
train данных

По-хорошему test должен быть размечен руками

Interval Methods in NLP → написал статью про последние события

program of thoughts

~~dangrebenkin~~ gpt auto augmentator

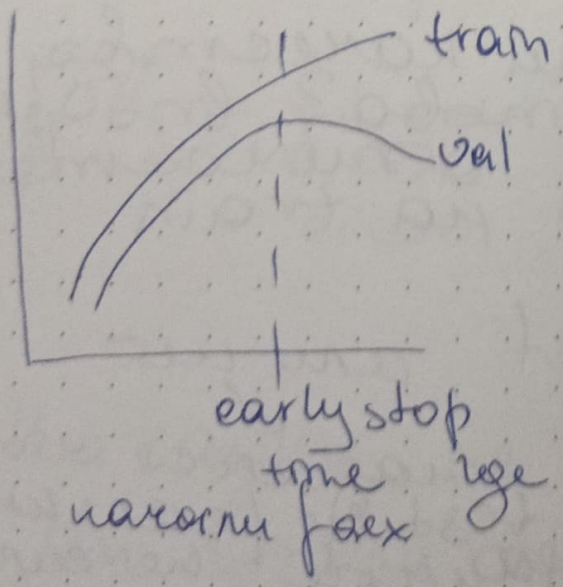
dangrebenkin

Переобучение избегать

→ Early stopping

train / val + test

после n итераций считаем L^* на val



где оптимум. до момента когда L^* на val достигла max → дальше будет оптимальнее

→ Кросс-валидация

один и тот же фактор не должен встретиться и в train и в val и вообще не должен в test

В train более старая дата, в test более новая

или одновременно слова

Анализ кач-ва бинарной классиф

TP, TN, FN, FP → Accuracy = $\frac{TP+FN}{TP+TN+FP+FN}$

Recall = $\frac{TP}{TP+FN}$

Precision = $\frac{TP}{TP+FP}$

Баланс типов ошибок

Чувствит = $\frac{TP}{TP+FN}$ · 100%

доля TP к общему числу Pпри-меров в обуч выборке

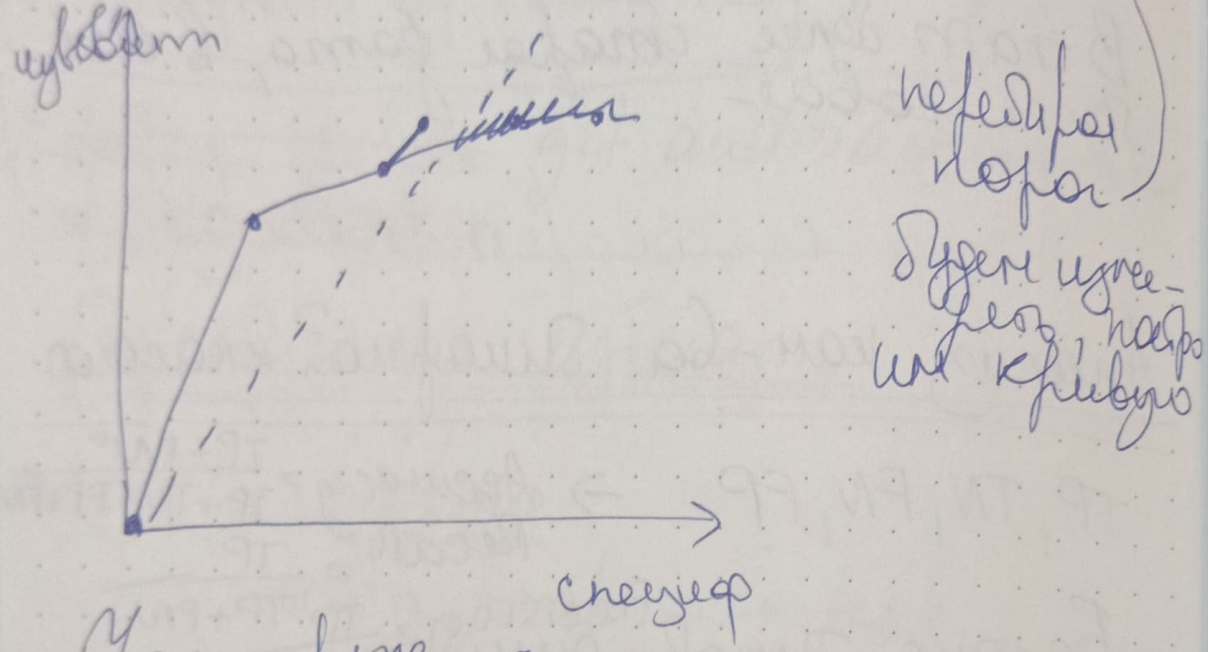
Специфичность = $\frac{TN}{TN+FP}$ · 100%.

насколько хорошо отключает N

В одно число хотелось бы

ROC кривая

$$y = \begin{cases} 1, & \frac{1}{1+e^{a+b_1 \cdot f_1(x) + \dots + b_n f_n(x)}} \geq th \\ 0, & \text{else} \end{cases}$$



Чем круче эта кривая - тем больше площадь под ней тем лучше наш алгоритм работает независимо от порога

Если по пунктирной линии - все плохо

Если ниже - так не бывает - что-то в алгоритме

Многоклассовая

рассматриваем как и бинарных классиф. и усред-няем рез-т

Точность и полнота

$$Precision = \frac{TP}{TP + FP}$$ не подменили ни один пример

$$Recall = \frac{TP}{TP + FN}$$ не забыли ни одного пример

К одному числу → F-мера

$$F = \frac{(\beta^2 + 1)\, Precision\, Recall}{\beta^2 Precision + Recall}$$

$\beta > 1$ → больше полнота

$0 < \beta < 1$ → больше точность

$\beta = 1$ → сбалансировано это F1

Многоклассовая —//—

Усреднение

1. TP_i, FP_i, FN_i для i-класса

2. Двоичное

Мажоритарное

Что брать?

→ если test сбалансир по классам, то давно

→ дисбаланс

↙ хотим более точно учесть вклады редких

↓ тасю

→ на частоты иные классы больше внимания

↓ тсю

Нормализация данных

Разные размеры диап данных

$$\begin{pmatrix} x_1(1) & \cdots & x_n(1) & y_1 \\ x_1(2) & & x_n(2) & y_2 \\ \vdots & & \vdots & \vdots \\ x_1(k) & \cdots & x_n(k) & y_k \end{pmatrix}$$

std - среднеквадратическое отклонение

mean - математическое ожидание

↓

$$\begin{pmatrix} \frac{x_1(1)-x_1(mean)}{x_1(std)} & \cdots & \frac{x_n(1)-x_n(mean)}{x_n(std)} & y_1 \\ \vdots & & \vdots & \vdots \\ \frac{x_1(k)-x_1(mean)}{x_1(std)} & \cdots & \frac{x_n(k)-x_n(mean)}{x_n(std)} & y_k \end{pmatrix}$$

TF, IDF и нормализ

ограниченный словарь
в 1 тексте только малая часть
этих слов
→ разреж знач

↳ спец: предобр: позиции
и значения ненул. эл-тов
sctpy Sparce

а если мы посчитаем
мат ожидание то поч-
ряем разреженность
чтобы использ MaxAbs Scaler
для 1 приз находим
максим и не учел
Но могут быть выбросы
Если исп MAxAbs Scaler
напрямую то отнормир по
знач. выбросов
↳ Robust Scaler нормир
с учет медианы. Опре-

дерем параметры диапазон и так нормируем

Чтобы не early stop а тренировать до сколько?

↳ Регуляризация

штраф за большие веса

Параметр C отвеч. как важен штраф. если (C > 1) или нужно модель до сложной (0 < C < 1)

Ridge regression

$$L^*(\theta) = C \cdot \frac{1}{K} \sum_{k=1}^{K} [y_k \ln(P_k) + (1-y_k)\ln(1-P_k)] - \frac{1}{2} \sum_{i=1}^{n} b_i^2$$

штрафуем плотные вектора

Lasso regression

штрафуем разреж. вектора

$$L^*(\theta) = C \frac{1}{K} \sum_{k=1}^{K} [y_k \ln(P_k) + (1-y_k)\ln(1-P_k)] * \ln(1-P_k)$$

В глуб. нейросетях лучше чем dropout